

プログラミングI 演習

Computer Programming I: Exercises

第4回 ループと数値計算

担当

A-P1 辻 愛里

A-P2 北 直樹

A-P3 清水 昭伸

1

本日の講義

2

本日の目標

- ループ文 (for文, while文) を覚える
- 簡単な流れ図 (フローチャート) が読めるようになる (再掲)
- 簡単な数値計算ができるようになる

本日の資料

1. 前々回 (第二回) と前回 (第三回) のおさらい
2. ループ文
3. 演習課題
4. デバッグとデバッガー

1. 前々回のおさらい

3

演習2-5の解答例

```
#include<stdio.h>
int main(void) {
    int n,m;
    printf("割られる数は?:");
    scanf("%d",&n);
    printf("割る数は?:");
    scanf("%d",&m);
    printf("%d÷%d=%dあまり%d¥n", n, m, n/m, n%m);
    return 0;
}
```

変数宣言

変数への値の入力

値の表示

- 変数の型に気を付けて書式(%d, %f,...)を指定する
- scanf文では変数名の前に「&」を必ずつける*
- n/mの結果が整数でなくても, int型の場合は整数のみ出力される

※第7回の「ポインタ」の時に詳しく説明します

1. 前々回のおさらい

4

発展2-1の解答例

```
#include<stdio.h>
int main(void)
{
    double d;
    int i;

    printf("小数を入力:");
    scanf("%lf",&d);

    d = d + 0.5;
    i = (int)d;
    printf("四捨五入すると%dです¥n", i);

    return 0;
}
```

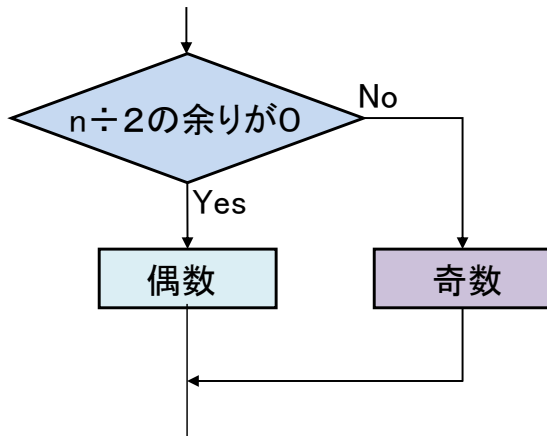
四捨五入のための処理

“(int)”は整数部のみを取り出す
処理. 型変換・キャスト.
(int) や (double)などの良く使う
ものは覚えておきましょう

四捨五入

+0.5

0.0 → 0	0.0 → 0.5
0.1 → 0	0.1 → 0.6
0.2 → 0	0.2 → 0.7
0.3 → 0	0.3 → 0.8
0.4 → 0	0.4 → 0.9
0.5 → 1	0.5 → 1.0
0.6 → 1	0.6 → 1.1
0.7 → 1	0.7 → 1.2
0.8 → 1	0.8 → 1.3
0.9 → 1	0.9 → 1.4
1.0 → 1	1.0 → 1.5
1.1 → 1	1.1 → 1.6
1.2 → 1	1.2 → 1.7
1.3 → 1	1.3 → 1.8
1.4 → 1	1.4 → 1.9
1.5 → 2	1.5 → 2.0
1.6 → 2	1.6 → 2.1
1.7 → 2	1.7 → 2.2
1.8 → 2	1.8 → 2.3
1.9 → 2	1.9 → 2.4

条件分岐 (if～else文)流れ図/Flow Chart

```
#include<stdio.h>
int main(void) {
    int n;
    printf("nを入力:");
    scanf("%d", &n);

    if (n%2 == 0) {
        printf("偶数です。¥n");
    } else {
        printf("奇数です。¥n");
    }
    return 0;
}
```

1. 前回のおさらい

演習3-5 うるう年判定

年数を入力(例: 2023)し, その年がうるう年かどうかを判定するプログラムを作成せよ.

うるう年:

1. 400で割り切れる年は, うるう年
2. 400で割り切れず, 100で割り切れる年は, うるう年ではない
3. 400でも100でも割り切れない場合, 4で割り切れる年は, うるう年, そうでない場合は, うるう年ではない

例:

- ・2016年 → うるう年
- ・2015年 → うるう年でない
- ・1900年 → うるう年でない
- ・2000年 → うるう年

if～else文を複数使うと
実現できます。
条件分岐の**順番**が大事!

演習3-5 うるう年判定 解答例: if~else文を複数使う

```

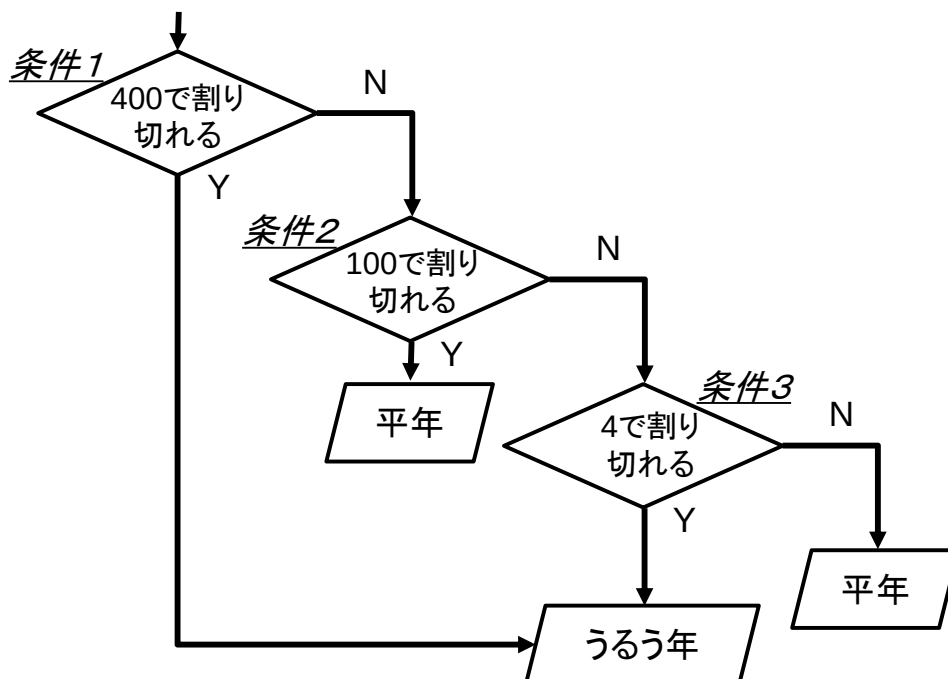
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);
    if( n%400==0 ) {
        printf("%d年はうるう年です。¥n", n);
    } else if( n%100==0 ) {
        printf("%d年はうるう年ではありません。¥n", n);
    } else if( n%4==0 ) {
        printf("%d年はうるう年です。¥n", n);
    } else {
        printf("%d年はうるう年ではありません。¥n", n);
    }
    return 0;
}

```

「==」は関係演算子
左辺と右辺が等しいか否かを判定するときに使う

条件1 {
条件2 {
条件3 {

演習3-5 うるう年判定



演習3-5 うるう年判定 別解: if~else文を1回だけ使う

```
#include <stdio.h>
int main(void){
    int n;
    scanf("%d", &n);
    if( n%4==0 && n%100!=0 || n%400==0 ){
        printf("%d年はうるう年です。¥n", n);
    } else {
        printf("%d年はうるう年ではありません。¥n", n);
    }
    return 0;
}
```

「&&」や「||」は論理演算子
→ 「かつ」や「または」を表現

①

②

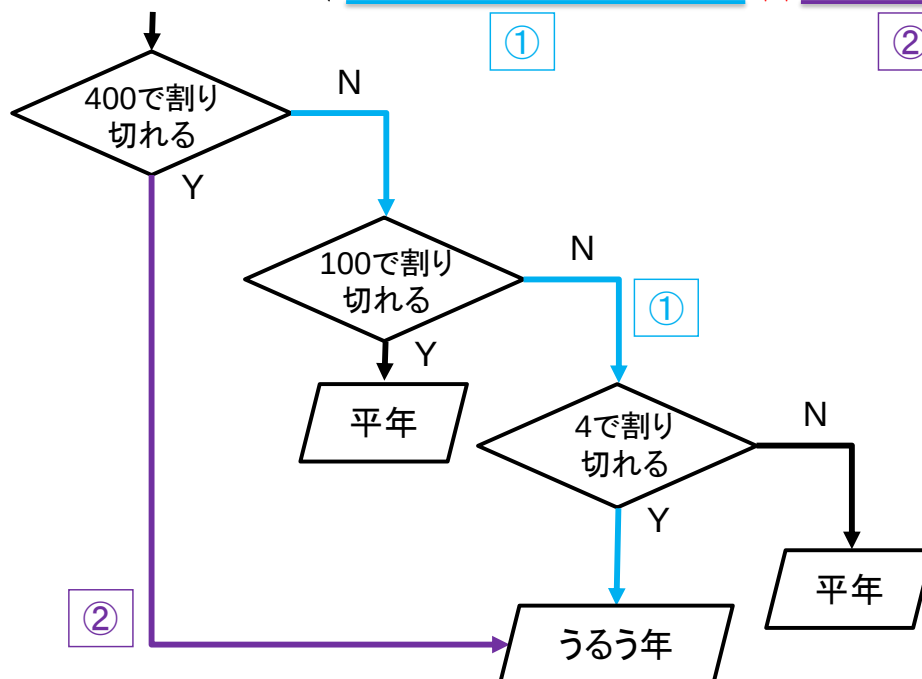
1. 前回のおさらい

演習3-5 うるう年判定

if (n%4==0 && n%100!=0 || n%400==0)

①

②



演習3-5 うるう年判定 別解: if~else文を1回だけ使う

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);
    if( n%4==0 && n%100!=0 || n%400==0 ) {
        printf("%d年はうるう年です。¥n", n);
    } else {
        printf("%d年はうるう年ではありません。¥n", n);
    }
    return 0;
}
```

(※実は、「&&」と「||」を一緒に使うと「&&」が先に処理されるという演算の優先順序ルールがあります。例えば、`A || B && C` は `A || (B && C)` として処理)

1. 前回のおさらい

演習3-5 うるう年判定 別解: if~else文を1回だけ使う

```
if( n%4==0 && n%100!=0 || n%400==0 ) {
    :
```

```
if( (n%4==0 && n%100!=0) || n%400==0 ) {
    :
```

プログラムの可読性を高め、バグの温床を避けるために、演算順序を明記すること、すなわち、**括弧内の演算が最も優先順位が高いことを利用し、括弧を使った明確な記述を強く推奨**

括弧をつけない記法でもコンパイルはできますが、括弧をつけることを検討するようwarningが出ることがあります。

note: place parentheses around the '&&' expression to silence this warning

```
if( n%4==0 && n%100!=0 || n%400==0 ){
```

発展3-1 関数の共有点の個数

a, b, c の3つの係数を入力し、関数 $y = ax^2 + bx + c$ と x 軸 ($y = 0$) との共有点の個数を調べるプログラムを作成せよ。例外処理(例: $a = 0$)も考慮してみよ。

二次方程式 $ax^2 + bx + c = 0$ の判別式 $D = b^2 - 4ac$ を利用する:

- $D > 0 \rightarrow$ 共有点2個
- $D = 0 \rightarrow$ 共有点1個
- $D < 0 \rightarrow$ 共有点0個

例外処理:

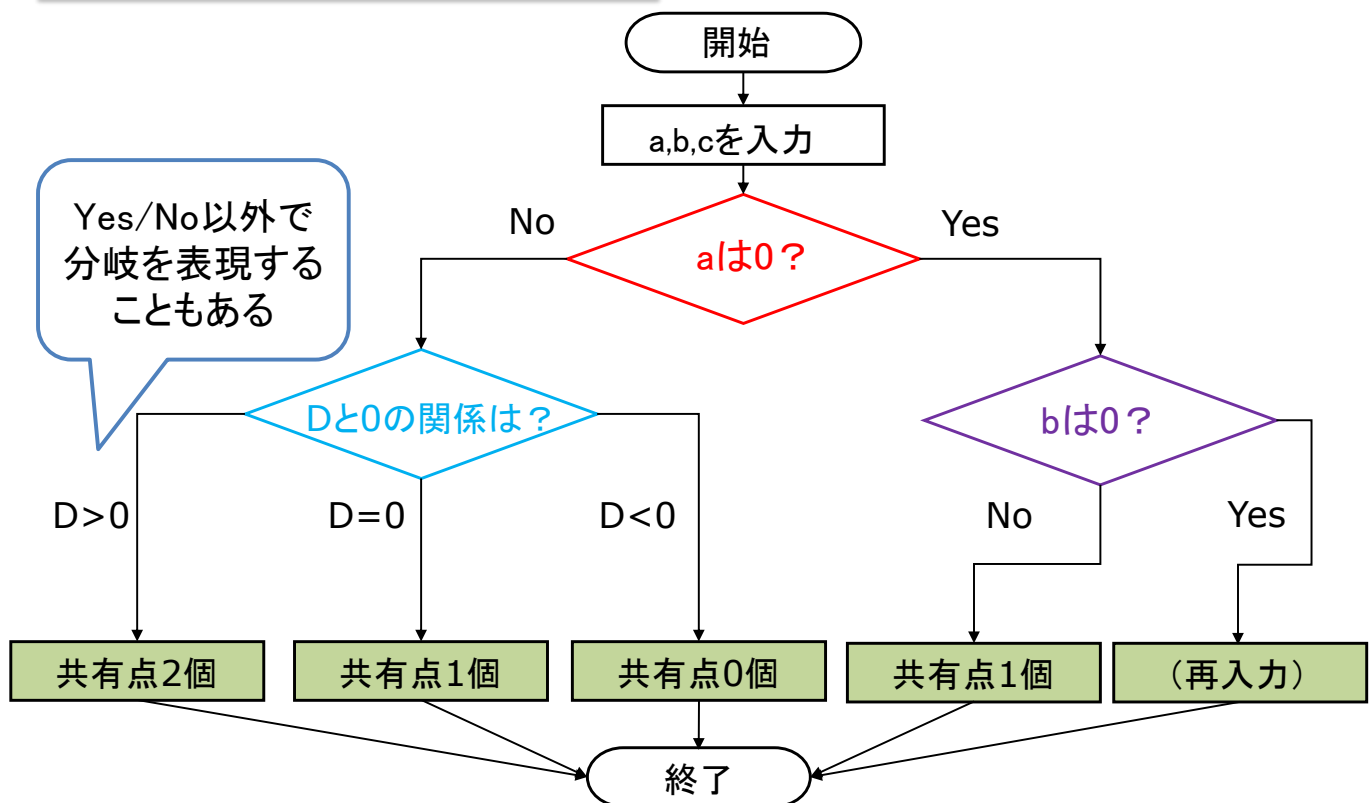
係数としてどんな値が入力された場合でも、最後まで実行できるようにすることが重要。

ヒント

$a=0$ の場合は常に $D \geq 0$ ですが、一次関数で共有点2個はおかしいですね。
 $a=0$ に加えて $b=0$ の場合は、さらに慎重な例外処理が必要です。

1. 前回のおさらい

発展3-1 関数の共有点の個数:フローチャート



発展3-1 関数の共有点の個数 解答例

```
#include <stdio.h>
int main(void){
    double a, b, c;
    double D;
    printf("aを入力:"); scanf("%lf", &a);
    printf("bを入力:"); scanf("%lf", &b);
    printf("cを入力:"); scanf("%lf", &c);
    D = b * b - 4 * a * c;

    if(a != 0){
        if(D > 0){ printf("共有点は2個です\n"); }
        else if(D == 0){ printf("共有点は1個です\n"); }
        else { printf("共有点は0個です\n"); }
    } else {
        if(b != 0){ printf("共有点は1個です\n"); }
        else { printf("再度入力してください\n"); }
    }
    return 0;
}
```

if～else文の内部に
if～else文がある
→ 入れ子構造

2. ループ文

繰り返し処理

- 特定の処理を繰り返す処理
- 工夫次第で、プログラムを短く簡潔に記述できる

1から10万までの数を表示するコード

ループのない世界

```
#include <stdio.h>
int main(void){
    printf("1\n");
    printf("2\n");
    (この間に10万行弱...)
    printf("100000\n");
    return 0;
}
```

ループのある世界

```
#include <stdio.h>
int main(void){
    int x;
    for(x=1; x<=100000; x++){
        printf("%d\n", x);
    }
    return 0;
}
```


for文

• 繰り返し処理を記述するための「制御構文」の1つ

1から100000までの数を表示するコード

```
for (x=1; x<=100000; x++)
{
    printf("%d¥n", x );
}
```

初期設定

ループの最初に実行

実行条件式

ループを繰り返す条件

繰り返し処理

ループで繰り返される処理

処理後実行文

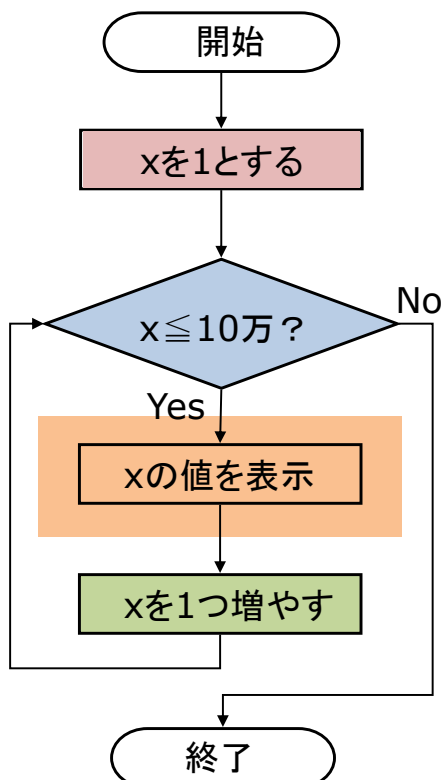
繰り返し処理後に実行

補足: 「x++」は 変数「x」を1つ増やす
という演算をします。
(この場合は、x=x+1と同じ結果になる)
「++」をインクリメント演算子と呼びます

2. ループ文

フローチャートとfor文の対応

1から100000までの数を表示する処理



```
for (x=1; x<=100000; x++)
{
    printf("%d¥n", x );
}
```

初期設定

ループの最初に実行

繰り返し処理

ループで繰り返される処理

実行条件式

ループを繰り返す条件

処理後実行文

繰り返し処理後に実行

while文

```
x=1;
while (x <= 100000)
{
    printf("%d¥n", x );
    x++;
}
```

1から100000までの数を表示する処理

```
for (x=1; x<=100000; x++)
{
    printf("%d¥n", x );
}
```

while文は・・・

実行条件式
ループを繰り返す条件

と

繰り返し処理
ループで繰り返される処理

の2つの部分から構成されている

for文における初期設定

while文の外に移動している

for文における処理後実行文

while文内の繰り返しの一部として最後に記述

2. ループ文

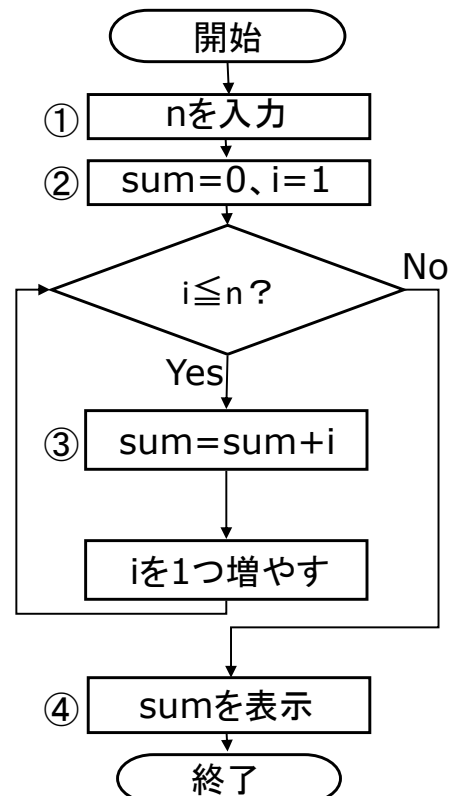
Q:1からnまでの和を求める

基本方針:

- ① 変数「n」をユーザーが入力
- ② 変数「sum」を初期化(0を代入)
- ③ カウンタ変数「i」を使い変数「sum」に加算
- ④ 合計を表示

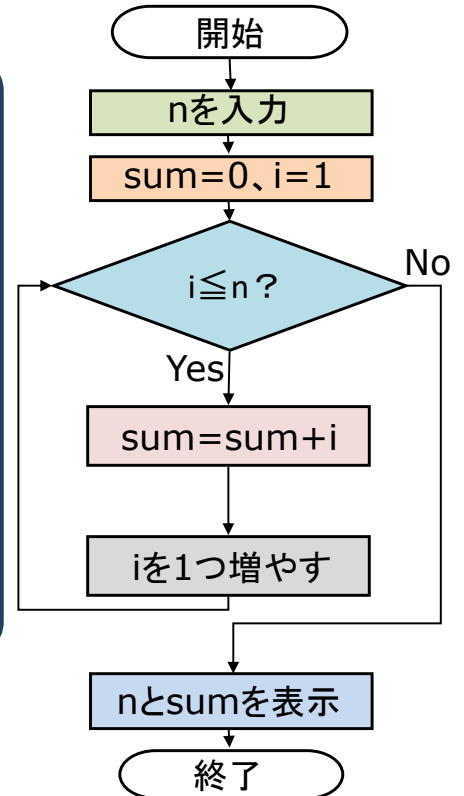
補足:カウンタ変数「i」

for文を使ったループを作成する場合、ループの回数をカウントするための変数が必要になる。カウンタ用の変数名を統一しておくプログラムが書きやすく/読みやすくなる。「i」、「j」、「k」などが良く使われる。原則として整数型を用いる。



A:1からnまでの和を求める

```
#include <stdio.h>
int main(void){
    int i,n,sum;
    printf("nを入力:");
    scanf("%d",&n);
    sum=0;
    for(i=1; i<=n; i++){
        sum=sum+i; /*和の計算*/
    }
    printf("1~%dの和は%d¥n",n,sum);
    return 0;
}
```



ここから演習課題です

演習4-1 for文の動作確認

```
#include <stdio.h>
int main(void) {
    int i, n, sum;

    printf("nを入力:");
    scanf("%d", &n);
    sum=0;
    for(i=1; i<=n; i++) {
        sum=sum+i;    /*和の計算*/
    }
    printf("1～%dの和は%d¥n", n, sum);
    return 0;
}
```

このとおりプログラムを書いてみて
for文の動作を確認しよう

演習4-2 while文の動作確認

```
#include <stdio.h>
int main(void) {
    int i, n, sum;

    printf("nを入力:");
    scanf("%d", &n);
    sum=0;
    for(i=1; i<=n; i++) {
        sum=sum+i;    /*和の計算*/
    }
    printf("1～%dの和は%d¥n", n, sum);
    return 0;
}
```

ループの部分をwhile文に書き換えて、
同じ処理(1～nまでの和を計算)
を実現しよう

3. 演習課題

25

演習4-3 数値計算①

n^2 を数列で表現した例

$$n^2 = \{1 + 3 + 5 + \cdots + 2n - 1\} = \sum_{i=1}^n (2i - 1)$$

合計を求めるための変数「ans」の変化

$ans_{(0)} = 0$	$= 0$
$ans_{(1)} = ans_{(0)} + 1$	$= 0 + 1$
$ans_{(2)} = ans_{(1)} + 3$	$= 0 + 1 + 3$
$ans_{(3)} = ans_{(2)} + 5$	$= 0 + 1 + 3 + 5$
$\vdots$	$\vdots$
$ans_{(n)} = ans_{(n-1)} + (2n - 1)$	$= 0 + 1 + \cdots + (2n - 1)$

変数「ans」が直前のループまでの和をもっている、その回の値だけを加えれば良い

・この式で本当に n^2 が計算できるか確かめてみよう
・正しい結果が得られる最大の n を求めてみよう

対応するプログラム

```
#include <stdio.h>
int main(void) {
    int n=10, i, ans=0;
    for(i=1; i<=n; i++) {
        ans=ans+(2*i-1);
    }
    printf("%dの2乗は%d\n", n, ans);
    return 0;
}
```

初項1, 公差2, 項数 n の等差数列の和
→ $(1/2) * \text{項数} * (\text{初項} + \text{末項})$
→ $(1/2) * n * (1 + (2n - 1)) = n^2$

3. 演習課題

26

演習4-4 数値計算②: 円周率の計算A

計算方法「グレゴリ・ライプニッツ級数」を用いる

$$4\left\{\frac{1}{1} - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} \cdots\right\} = 4 \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} = \pi$$

基本方針

- ・ Σ の型式になっているので、演習4-3を基本とすればよさそう
- ・ $(-1)^n$ はループ1回毎に符号が入れ替わっているだけ
(if文を用いるか、ある変数「a=-1.0」の符号を毎回入れ替える等)
- ・ まずは Σ 以降の部分を実行し、最後に4を掛ける

問題点

- ・ $0 \sim \infty$ までのループでは終わりが来ない → **回数か精度を決める**

円周率: 3.14159 26535 89793 23846 26433 83279 50288

繰り返し回数を適当に決めて実行し、円周率に近づくか確かめてみよう

3. 演習課題

27

演習4-5 数値計算③: 円周率の計算B

繰り返し回数を適当に決めて実行し, 円周率に近づくか確かめてみよう

「マチンの公式」を用いると
少ない反復回数で π を求めることができる

$$16\left\{\frac{1}{5} - \frac{1}{3 \times 5^3} + \frac{1}{5 \times 5^5} - \frac{1}{7 \times 5^7} \cdots\right\} \\ - 4\left\{\frac{1}{239} - \frac{1}{3 \times 239^3} + \frac{1}{5 \times 239^5} - \frac{1}{7 \times 239^7} \cdots\right\} = \\ 16 \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1) \times 5^{(2n+1)}} - 4 \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1) \times 239^{(2n+1)}} = \pi$$

239の〇乗はかなり値が大きい
nが小さくても, きちんと計算できないかも

円周率: 3.14159 26535 89793 23846 26433 83279 50288

3. 演習課題

28

発展4-1 精度の測定

グレゴリ・ライプニッツ級数 と マチンの公式 の精度比較

「マチンの公式」を用いると
少ない反復回数で π を求めることができる



これが本当に正しいか否か確かめてみよう

基本方針

- ある繰り返し回数に対して, グレゴリ・ライプニッツ級数を用いた結果と, マチンの公式を用いた結果を出力できるようにする
- 繰り返し回数を1から適当なところまで1ずつ増やし, 値の変化を確認する
- (上級者向け) 1から入力した繰り返し回数までの結果すべてを出力する
→ 比較しやすい. どちらが先に円周率に近づくかが一目でわかる

計算の打ち切り

計算機では有限の数しか扱うことが出来ない



ある程度のところで妥協して、計算を打ち切る必要がある

打ち切り方法

①計算量に基づいた方法

ループを一定の回数で打ち切る
簡単だが、誤差がどの程度出るかは分からない(数列によって変わる)

②精度に基づいた方法

ループ毎に和を求める変数の変化量を監視し、
一定値以下の変化しか起きなくなった場合に計算を打ち切る
計算時間が一定ではない(終わらないこともある)

本日の説明内容はおしまいです。
続いて、デバッグとデバッガーについて補足説明をします。
その後、課題に取り組んでください。

• デバッグとは

- プログラム実行中にどういった処理が行われているか分析し挙動を詳細に理解すること
(例えばどのように値が変化していくか, どのような順序で関数が呼び出されるか, どの箇所で計算時間がかかるか※, といったこと)
- また分析で明らかになった知見を元に, 改善すべき箇所を特定し訂正を加えること

[Debugging with gdb, tenth edition, 2019]

<https://sourceware.org/gdb/onlinedocs/gdb.pdf.gz>

• デバグガーとは

デバッグを効率よく進めるために, 様々な支援を行うツール

※計算負荷といった性能の分析はプロファイリングと呼んで区別することもある 31

4. デバッグとデバグガー: 解決すべきエラーについて 32



①コンパイルエラー(compile-time error): コンパイルに失敗すること
(主な対処法)コンパイラによるエラーメッセージや, エディタの支援機能を参考に
対処

②ランタイムエラー(runtime error): コンパイルは成功したが,
実行時に失敗が生じること
(主な対処法)デバグガーを使い, エラー原因を特定して対処

関連

動的テスト: 動作結果に基づく不具合検査

静的テスト: 動作させずに行うコードの文法等の検査

4. デバッグとデバッガー：デバッグ不足と事故 33

ランタイムエラーの例：桁あふれによる欧州宇宙機構ロケットAriane 5 爆発事故(1996年, 死者0)



<https://www.youtube.com/watch?v=WFZwl10HEsw>

```
signed short int x=32760;
for (int i = 0; i < 10; i++)
{
    x++;
    printf("%d: %d\n", i, x);
}
```

signed short int型変数が扱えるのは- 32768～32767
(誤った変数型の使用が原因の事故)

コンパイルエラーは生じない

桁あふれ
(overflow)

0:	32761
1:	32762
2:	32763
3:	32764
4:	32765
5:	32766
6:	32767
7:	-32768
8:	-32767
9:	-32766

4. デバッグとデバッガー：デバッグ不足と事故 34

その他の事例

2005.11.15 TUE 01:00

WIRED

「史上最悪のソフトウェアバグ」ワースト10を紹介(上)

1982年に旧ソ連でシベリアを横断するガス・パイプラインが大爆発を起こしたのは、管理用に購入されたカナダ製コンピューターシステムに、米中央情報局(CIA)のスパイがバグを仕掛けたからだと言われる。放射線治療装置がプログラムミスによる誤動作を起こし、死者が出たケースもある。――ワイアード・ニュースが選ぶバグの歴代ワースト10。

「史上最悪のソフトウェアバグ」ワースト10を紹介(下)

1993年、米インテル社の初期『Pentium』チップは特定の浮動小数点数の除算で間違った答えを出すことが明らかになり、交換のための損害は約4億7500万ドルに上った。ほかに、『アリアン5』ロケットを空中分解させたバグなど、ワイアード・ニュースが選ぶバグの歴代ワースト10、後半。

<https://wired.jp/2005/11/15/%E3%80%8C%E5%8F%B2%E4%B8%8A%E6%9C%80%E6%82%AA%E3%81%AE%E3%82%BD%E3%83%95%E3%83%88%E3%82%A6%E3%82%A2%E3%83%90%E3%82%B0%E3%80%8D%E3%83%AF%E3%83%BC%E3%82%B9%E3%83%88%E3%82%92%E7%B4%B9%E4%BB%8B/>

4. デバッグとデバッガー: コンパイルエラーの例 35

ソースコード

```
#include <stdio.h>

int main (void){
    int a[10]
    for (int i=1; i<20; i++){
        a[i]=10;
        printf("i: %d, a[i]: %d ¥n", i, a[i]);
    }
    return 0;
}
```

セミコロン忘れ

実行結果

```
simiz@DESKTOP-3ITML0G:/mnt/c/Users/as/Documents/プログラミング 演習$ gcc ./p3.c
./p3.c: In function 'main':
./p3.c:5:1: error: expected '=', ',', ';', 'asm' or '__attribute__' before 'for'
  5 | for(int i=1; i<20; i++){
    | ^~~
./p3.c:6:5: error: 'a' undeclared (first use in this function)
  6 |     a[i]=10;
    |     ^
./p3.c:6:5: note: each undeclared identifier is reported only once for each function it appears in
```

「“for”の前に、=や、や；といった記号が入ることが期待されます」

「aが宣言されていません」というエラーメッセージ

35

4. デバッグとデバッガー: ランタイムエラーの例 1 36

ソースコード

```
#include <stdio.h>

int main (void){
    int a[10];
    for (int i=1; i<20; i++){
        a[i]=10;
        printf("i: %d, a[i]: %d ¥n", i, a[i]);
    }
    return 0;
}
```

セミコロンを訂正

実行結果

```
i:12, a[i]:10
i:13, a[i]:10
i:14, a[i]:10
i:15, a[i]:10
i:16, a[i]:10
i:17, a[i]:10
i:18, a[i]:10
i:19, a[i]:10
*** stack smashing detected ***: terminated
Aborted (core dumped)
```

配列外参照エラー

1次元配列a(サイズ10)の
10以上のインデックスに
アクセス

36

4. デバッグとデバッガー: ランタイムエラーの例2³⁷

ソースコード

p2.c

```
int x=0;
if ( x != 0 );
{
    printf("x is not zero.");
}
```

実行結果

```
PS C:\Users\Public\vscode> .\a.exe
x is not zero.
```

x=0で初期化されているので, if 条件式 (x!=0) はFalseのはず.
if文の本体は実行されないはずだが, x is not zero. と出力される

```
if ( x != 0 );
```

原因はこのセミコロン(本来は不要)

37

4. デバッグとデバッガー: C言語用デバッガ

38

様々なデバッガ

【GDB】C, C++, Objective-C, Java, FORTRAN等の言語で記述したソースコードの実行動作のデバッグ支援. 愛好者が多い.

【LLDB】C, C++, Objective-C言語のデバッグ支援.
Mac OSX Xcodeで, 標準デバッガとして採用されている.

様々なデバッガ

【DDD】GDBを視覚的に操作することを支援するソフトウェア

【VS Codeのデバッガ】DDDと同じく, GDBやLLDBといったデバッガを視覚的に操作するためのソフトウェア

Win/LinuxはGDB, MacはGDB/LLDBを指定して利用する

それぞれ得手不得手があり, それぞれ強みがあります.
講義ではVS Codeデバッガから入門します

https://code.visualstudio.com/docs/languages/cpp#_debugging³⁹

4. デバッグとデバッガー: 基本的な考え方 40



1. 実行時に, 異常な挙動が生じることに気づいた
2. 異常な挙動の前後で, どのように変数の値が変化していくかなどを, 詳細に観察

実行中の挙動(ランタイムエラー)に興味がありますので
ユーザは実行可能ファイルを分析していくことになります.

ソースコードを変更したら, そのたびに実行可能ファイルにコンパイルしてから
デバッガで分析する必要があります

4. デバッグとデバッガー: 基本的な考え方 41

コードを少しずつ実行し, 誤った実装になっている箇所を発見することが最も基本的な対処

手軽な手法: printfにより値を出力する手法 (通称 printデバッグ※)

```
signed short int x=32760;
for (int i = 0; i < 10; i++)
{
    x++;
    printf("%d: %d¥n", i, x);
}
```



0:	32761
1:	32762
2:	32763
3:	32764
4:	32765
5:	32766
6:	32767
7:	-32768
8:	-32767
9:	-32766

桁あふれ
(overflow) ↓

※変数値の変化を追いかける技法は, 総称して"トレース"と呼ばれます

41

4. デバッグとデバッガー: 基本的な考え方 42

printデバッグは手軽な一方, デメリットもあります

- 長いwhile ループやfor ループ内でprint出力する場合膨大な文字が高速で出力されるため, 目視できない
- printf文を書き加えたり, コメントアウトする必要がある, 新たなバグの温床になる危険性がある
- etc

以上のような理由が, デバッガを使うモチベーションです

42

先ほどランタイムエラーとして提示した例を使って、
コンパイルメッセージに注目したデバッグの事例を紹介します

p2.c

```
int x=0;
if ( x != 0 );
{
    printf("x is not zero.");
}
```



```
PS C:\Users\Public\vscode> .\a.exe
x is not zero.
```

x=0で初期化されているので、if 条件式 (x!=0)はFalse.
ところが x is not zero. と出力される

```
if ( x != 0 );
```

原因はこのセミicolon(本来は不要)

43

コンパイル時に得られるwarningメッセージは無視せずきちんと読みましょう

コンパイル環境① gcc(Apple LLVM version 10.0.1)

```
~/workspace_VS/lecture1 (master●●)
$ gcc p2.c
p2.c:6:17: warning: if statement has empty body [-Wempty-body]
    if ( x != 0 );
    ^
```

「if文の本体がemptyになっています」という警告(warning)

コンパイル環境② gcc (ver. 8.2.0, MinGW): warningメッセージ無し

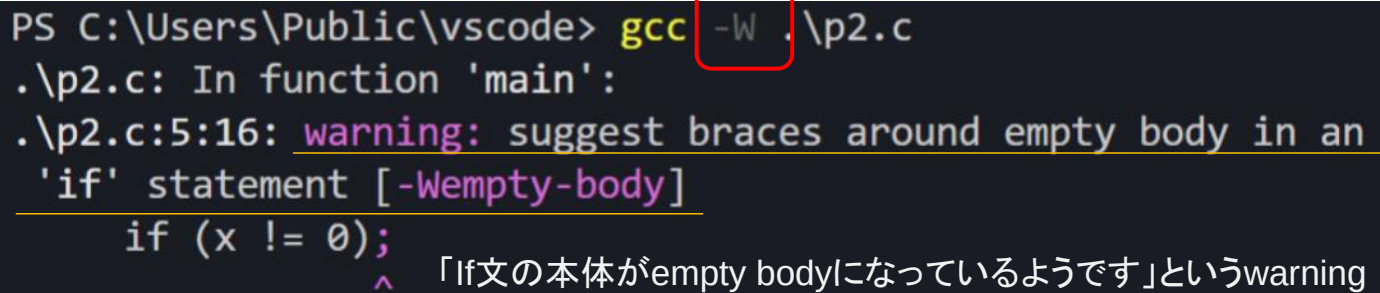
```
PS C:\Users\Public\vscode> gcc .\p2.c
PS C:\Users\Public\vscode> .\a.exe
x is not zero.
```

②のように、コンパイラのバージョン(や個々人の設定)によって
error/warningメッセージ が出ないこともあるので注意

44

ランタイムエラーの例2

コンパイル環境② gcc (ver. 8.2.0, MinGW) でも
コンパイル・オプション (compile option `-W`) をつけるとwarningメッセージが得られる



```
PS C:\Users\Public\vscode> gcc -W ./p2.c
.\p2.c: In function 'main':
.\p2.c:5:16: warning: suggest braces around empty body in an
'if' statement [-Wempty-body]
    if (x != 0);
    ^          ^
「If文の本体がempty bodyになっているようです」というwarning
```

ゆっくりとで良いので、自身が使用しているコンパイラの癖も
把握できるようになっていきましょう

45

4. デバッグとデバッガー: VS code の場合

- 以下の別ファイルを参照すること.
 - winOSデバッグ環境構築_2023.pdf
 - macOSデバッグ環境構築_2023.pdf

※math.hをincludeした際にうまく動作しない場合には、
以下のtasks.jsonとlaunch.jsonを使ってください。

- tasks.json
- launch.json

- 超初心者でもわかるデバッグ方法
<https://docs.microsoft.com/ja-jp/visualstudio/debugger/debugging-absolute-beginners?view=vs-2019>
- VS codeにおけるC言語デバッガ
https://code.visualstudio.com/docs/languages/cpp#_debugging